

Gradient Descent and Logistic Regression

Binary Logistic Regression, Gradient Math, and Multiclass Softmax Regression

CSE427: Machine Learning

Lecture Roadmap

Today we connect four parts:

- 1 Gradient descent basics
- 2 Gradient descent math with a solved linear model example
- 3 Binary logistic regression review
- 4 Multiclass logistic regression using softmax

Big Picture

Gradient descent is the learning method. Logistic regression is the model.

Why Do We Need Gradient Descent?

In machine learning, a model has unknown parameters.

Example:

$$\hat{z} = ax + by + c$$

Here:

$$a, \quad b, \quad c$$

are unknown parameters.

Training Goal

Find parameter values that make predictions close to the actual target values.

To do this, we define a loss function and minimize it.

Loss Function

A loss function measures how wrong the model is.

For regression, a common loss is:

$$SSR = \sum_{i=1}^m (z_i - \hat{z}_i)^2$$

where:

z_i = actual value

$\hat{z}_i$ = predicted value

Meaning

Smaller SSR means better prediction.

Gradient Descent Intuition

Gradient descent updates parameters step by step.

$$\text{New parameter} = \text{Old parameter} - \alpha \times \text{Gradient}$$

For parameter a :

$$a := a - \alpha \frac{\partial SSR}{\partial a}$$

For parameter b :

$$b := b - \alpha \frac{\partial SSR}{\partial b}$$

For parameter c :

$$c := c - \alpha \frac{\partial SSR}{\partial c}$$

Why Do We Subtract the Gradient?

The gradient points toward the direction where loss increases fastest.
So to reduce the loss, we move in the opposite direction.

$$\theta_{new} = \theta_{old} - \alpha \frac{\partial J}{\partial \theta}$$

Term	Meaning
θ	Any parameter
α	Learning rate
$\frac{\partial J}{\partial \theta}$	Slope or gradient

Important

If gradient is negative, subtracting it increases the parameter.

Solved Gradient Example: Dataset

Suppose we have the following data:

x	y	z
1	2	13
2	3	18
3	4	23

Assume the prediction equation:

$$\hat{z} = ax + by + c$$

We want to learn:

$$a, \quad b, \quad c$$

using gradient descent.

Prediction Equations

For each row:

$$\hat{z}_1 = a(1) + b(2) + c = a + 2b + c$$

$$\hat{z}_2 = a(2) + b(3) + c = 2a + 3b + c$$

$$\hat{z}_3 = a(3) + b(4) + c = 3a + 4b + c$$

Errors:

$$e_1 = 13 - (a + 2b + c)$$

$$e_2 = 18 - (2a + 3b + c)$$

$$e_3 = 23 - (3a + 4b + c)$$

$$SSR = e_1^2 + e_2^2 + e_3^2$$

So:

$$SSR = \{13 - (a + 2b + c)\}^2$$

$$+ \{18 - (2a + 3b + c)\}^2$$

$$+ \{23 - (3a + 4b + c)\}^2$$

Goal

Find a, b, c that minimize this SSR.

Gradient with Respect to a

$$\frac{\partial SSR}{\partial a} = 2e_1 \frac{\partial e_1}{\partial a} + 2e_2 \frac{\partial e_2}{\partial a} + 2e_3 \frac{\partial e_3}{\partial a}$$

Since:

$$e_1 = 13 - (a + 2b + c), \quad \frac{\partial e_1}{\partial a} = -1$$

$$e_2 = 18 - (2a + 3b + c), \quad \frac{\partial e_2}{\partial a} = -2$$

$$e_3 = 23 - (3a + 4b + c), \quad \frac{\partial e_3}{\partial a} = -3$$

Therefore:

$$\frac{\partial SSR}{\partial a} = -2(1)e_1 - 2(2)e_2 - 2(3)e_3$$

Gradient with Respect to b and c

For b :

$$\frac{\partial SSR}{\partial b} = -2(2)e_1 - 2(3)e_2 - 2(4)e_3$$

For c :

$$\frac{\partial SSR}{\partial c} = -2e_1 - 2e_2 - 2e_3$$

Reason

The coefficient of the parameter inside the prediction decides the derivative.

For example:

$$\frac{\partial}{\partial b}(a + 2b + c) = 2$$

Initial Values

Let:

$$a = 1, \quad b = 1, \quad c = 1$$

Then predictions:

$$\hat{z}_1 = 1 + 2(1) + 1 = 4$$

$$\hat{z}_2 = 2(1) + 3(1) + 1 = 6$$

$$\hat{z}_3 = 3(1) + 4(1) + 1 = 8$$

Errors:

$$e_1 = 13 - 4 = 9$$

$$e_2 = 18 - 6 = 12$$

Compute Gradients

For a :

$$\begin{aligned}\frac{\partial SSR}{\partial a} &= -2(1)(9) - 2(2)(12) - 2(3)(15) \\ &= -18 - 48 - 90 = -156\end{aligned}$$

For b :

$$\begin{aligned}\frac{\partial SSR}{\partial b} &= -2(2)(9) - 2(3)(12) - 2(4)(15) \\ &= -36 - 72 - 120 = -228\end{aligned}$$

For c :

$$\frac{\partial SSR}{\partial c} = -2(9) - 2(12) - 2(15)$$

$$= -18 - 24 - 30 = -72$$

Update Parameters

Let learning rate:

$$\alpha = 0.001$$

Update rule:

$$a := a - \alpha \frac{\partial SSR}{\partial a}$$

$$b := b - \alpha \frac{\partial SSR}{\partial b}$$

$$c := c - \alpha \frac{\partial SSR}{\partial c}$$

Substitute:

$$a = 1 - 0.001(-156) = 1.156$$

$$b = 1 - 0.001(-228) = 1.228$$

$$c = 1 - 0.001(-72) = 1.072$$

$a = 1.156, \quad b = 1.228, \quad c = 1.072$

Check New Prediction After One Update

New model:

$$\hat{z} = 1.156x + 1.228y + 1.072$$

For row 1:

$$\begin{aligned}\hat{z}_1 &= 1.156(1) + 1.228(2) + 1.072 \\ &= 1.156 + 2.456 + 1.072 = 4.684\end{aligned}$$

Old prediction was:

4

Actual value:

13

Observation

Prediction moved upward toward the actual value.

Binary Logistic Regression: Motivation

For binary classification:

$$y \in \{0, 1\}$$

Example:

$$y = 1 \Rightarrow \text{Pass}$$

$$y = 0 \Rightarrow \text{Fail}$$

A linear model gives:

$$z = w_1x_1 + w_2x_2 + b$$

But z can be any real number.

We need probability:

$$0 \leq P(y = 1|x) \leq 1$$

Sigmoid Function

$$\hat{y} = P(y = 1|x) = \sigma(z)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Important:

$$z = 0 \Rightarrow \sigma(z) = 0.5$$

$$z > 0 \Rightarrow \sigma(z) > 0.5$$

$$z < 0 \Rightarrow \sigma(z) < 0.5$$

Binary Logistic Prediction Rule

$$\hat{y} = P(y = 1|x)$$

Common threshold:

$$\hat{y} \geq 0.5 \Rightarrow \text{Predict 1}$$

$$\hat{y} < 0.5 \Rightarrow \text{Predict 0}$$

The decision boundary is:

$$z = 0$$

or:

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = 0$$

Binary Cross-Entropy Loss

For one example:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

If $y = 1$:

$$L = -\log(\hat{y})$$

If $y = 0$:

$$L = -\log(1 - \hat{y})$$

Meaning

Loss is small when the model gives high probability to the correct class.

Binary Logistic Gradient

For one example:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$\hat{y} = \sigma(z)$$

The gradients are:

$$\frac{\partial L}{\partial w_j} = (\hat{y} - y)x_j$$

$$\frac{\partial L}{\partial b} = \hat{y} - y$$

Update rules:

$$w_j := w_j - \eta(\hat{y} - y)x_j$$

$$b := b - \eta(\hat{y} - y)$$

Key Term

$$\hat{y} - y$$

is the prediction error.

OR Gate Dataset

We want to learn a classifier for OR gate.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Initial model:

$$z = w_1x_1 + w_2x_2 + b$$

Given:

$$w_1 = 0.5, \quad w_2 = 0.75, \quad b = -1.25$$

Learning rate:

$$\eta = 0.2$$

OR Gate: Example 1

Input:

$$x_1 = 0, \quad x_2 = 0, \quad y = 0$$

Compute:

$$z = 0.5(0) + 0.75(0) - 1.25 = -1.25$$

$$\hat{y} = \sigma(-1.25) = 0.22$$

Prediction:

$$\hat{y} < 0.5 \Rightarrow h_w(x) = 0$$

Actual:

$$y = 0$$

Correct prediction, no update

OR Gate: Example 2

Input:

$$x_1 = 0, \quad x_2 = 1, \quad y = 1$$

Current parameters:

$$w_1 = 0.5, \quad w_2 = 0.75, \quad b = -1.25$$

$$z = 0.5(0) + 0.75(1) - 1.25 = -0.50$$

$$\hat{y} = \sigma(-0.50) = 0.38$$

Prediction:

$$0.38 < 0.5 \Rightarrow h_w(x) = 0$$

But actual:

$$y = 1$$

So update is needed.

OR Gate: Update After Example 2

Use update:

$$w_j := w_j + \eta(y - \hat{y})x_j$$

$$b := b + \eta(y - \hat{y})$$

Here:

$$y - \hat{y} = 1 - 0.38 = 0.62$$

$$w_1 = 0.5 + 0.2(0.62)(0) = 0.5$$

$$w_2 = 0.75 + 0.2(0.62)(1) = 0.874$$

$$b = -1.25 + 0.2(0.62) = -1.126$$

Approx:

OR Gate: Example 3

Input:

$$x_1 = 1, \quad x_2 = 0, \quad y = 1$$

Current model:

$$z = 0.50x_1 + 0.87x_2 - 1.13$$

$$z = 0.50(1) + 0.87(0) - 1.13 = -0.63$$

$$\hat{y} = \sigma(-0.63) = 0.35$$

Prediction:

$$0.35 < 0.5 \Rightarrow h_w(x) = 0$$

Actual:

$$y = 1$$

So update is needed.

OR Gate: Update After Example 3

$$y - \hat{y} = 1 - 0.35 = 0.65$$

$$w_1 = 0.50 + 0.2(0.65)(1) = 0.63$$

$$w_2 = 0.87 + 0.2(0.65)(0) = 0.87$$

$$b = -1.13 + 0.2(0.65) = -1.00$$

Updated model:

$$z = 0.63x_1 + 0.87x_2 - 1.00$$

OR Gate: Example 4

Input:

$$x_1 = 1, \quad x_2 = 1, \quad y = 1$$

Current model:

$$z = 0.63x_1 + 0.87x_2 - 1.00$$

$$z = 0.63(1) + 0.87(1) - 1.00 = 0.50$$

$$\hat{y} = \sigma(0.50) = 0.62$$

Prediction:

$$0.62 \geq 0.5 \Rightarrow h_w(x) = 1$$

Actual:

$$y = 1$$

Correct prediction, no update

OR Gate After First Cycle

After one full cycle:

$$w_1 = 0.63$$

$$w_2 = 0.87$$

$$b = -1.00$$

Final model after first cycle:

$$z = 0.63x_1 + 0.87x_2 - 1.00$$

Observation

The model has moved closer to OR gate behavior.

OR Gate: Second Cycle Example

Use:

$$z = 0.63x_1 + 0.87x_2 - 1.00$$

For:

$$x_1 = 0, \quad x_2 = 1, \quad y = 1$$

$$z = 0.63(0) + 0.87(1) - 1.00 = -0.13$$

$$\hat{y} = \sigma(-0.13) = 0.47$$

Prediction:

$$0.47 < 0.5 \Rightarrow h_w(x) = 0$$

Actual:

$$y = 1$$

So update is needed.

OR Gate: Second Cycle Update

$$y - \hat{y} = 1 - 0.47 = 0.53$$

$$w_1 = 0.63 + 0.2(0.53)(0) = 0.63$$

$$w_2 = 0.87 + 0.2(0.53)(1) = 0.976$$

$$b = -1.00 + 0.2(0.53) = -0.894$$

So a corrected update gives:

$$z = 0.63x_1 + 0.976x_2 - 0.894$$

Note

Always recompute carefully; rounding errors can change the displayed values.

Why Multiclass Logistic Regression?

Binary logistic regression handles:

$$y \in \{0, 1\}$$

But many problems have more than two classes.

Examples:

$$\text{Image class} \in \{\text{Cat}, \text{Dog}, \text{Bird}\}$$

$$\text{Grade} \in \{A, B, C, D, F\}$$

$$\text{Disease type} \in \{C_1, C_2, C_3, \dots, C_k\}$$

Need

We need probabilities for all classes, not only class 0 and class 1.

One Score for Each Class

For K classes, compute K scores.

For class 1:

$$z_1 = w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n + b_1$$

For class 2:

$$z_2 = w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n + b_2$$

Continue until class K :

$$z_K = w_{K1}x_1 + w_{K2}x_2 + \cdots + w_{Kn}x_n + b_K$$

Meaning

Each class has its own linear classifier.

Why Not Use Sigmoid Separately?

If we apply sigmoid separately:

$$\sigma(z_1), \quad \sigma(z_2), \quad \sigma(z_3)$$

the probabilities may not sum to 1.

Example:

$$0.8 + 0.7 + 0.6 = 2.1$$

This cannot be a probability distribution.

Solution

Use softmax.

Softmax converts all scores into probabilities that sum to 1.

Softmax Function

Given scores:

$$z_1, z_2, \dots, z_K$$

Softmax probability for class i :

$$P(y = i|x) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Vector form:

$$\text{softmax}(\mathbf{z}) = \left[\frac{e^{z_1}}{\sum_j e^{z_j}}, \frac{e^{z_2}}{\sum_j e^{z_j}}, \dots, \frac{e^{z_K}}{\sum_j e^{z_j}} \right]$$

Important

All softmax outputs are between 0 and 1 and sum to 1.

Softmax Solved Example: Scores

Suppose we have 6 class scores:

$$z_1 = 0.6, \quad z_2 = 1.1, \quad z_3 = -1.5$$

$$z_4 = 1.2, \quad z_5 = 3.2, \quad z_6 = -1.1$$

We compute:

$$P(y = i|x) = \frac{e^{z_i}}{\sum_{j=1}^6 e^{z_j}}$$

First compute exponentials:

$$e^{0.6} = 1.82, \quad e^{1.1} = 3.00, \quad e^{-1.5} = 0.22$$

$$e^{1.2} = 3.32, \quad e^{3.2} = 24.53, \quad e^{-1.1} = 0.33$$

Softmax Solved Example: Denominator

Denominator:

$$S = e^{0.6} + e^{1.1} + e^{-1.5} + e^{1.2} + e^{3.2} + e^{-1.1}$$

$$S = 1.82 + 3.00 + 0.22 + 3.32 + 24.53 + 0.33$$

$$S = 33.22$$

Now probabilities:

$$P(y_1|x) = \frac{1.82}{33.22} = 0.055$$

$$P(y_2|x) = \frac{3.00}{33.22} = 0.090$$

$$P(y_3|x) = \frac{0.22}{33.22} = 0.007$$

Softmax Solved Example: Final Probabilities

Continue:

$$P(y_4|x) = \frac{3.32}{33.22} = 0.100$$

$$P(y_5|x) = \frac{24.53}{33.22} = 0.738$$

$$P(y_6|x) = \frac{0.33}{33.22} = 0.010$$

So:

$$\hat{\mathbf{y}} = [0.055, 0.090, 0.007, 0.100, 0.738, 0.010]$$

Predicted class:

y_5

because it has the highest probability.

Multiclass Cross-Entropy Loss

For multiclass classification:

$$L = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

where y_k is one-hot encoded.

Example:

If actual class is y_2 , then:

$$\mathbf{y} = [0, 1, 0, 0, 0, 0]$$

So:

$$L = -\log(\hat{y}_2)$$

If:

$$\hat{y}_2 = 0.090$$

Why Loss is Large Here

Prediction:

$$\hat{\mathbf{y}} = [0.055, 0.090, 0.007, 0.100, 0.738, 0.010]$$

Predicted class:

y_5

Actual class:

y_2

The model gives only:

0.090

probability to the correct class.

Therefore:

Multiclass Gradient

For softmax regression, the gradient has the same clean form.

For class k and feature i :

$$\frac{\partial L}{\partial w_{ki}} = (\hat{y}_k - y_k)x_i$$

Update rule:

$$w_{ki} := w_{ki} - \eta(\hat{y}_k - y_k)x_i$$

Equivalent form:

$$w_{ki} := w_{ki} + \eta(y_k - \hat{y}_k)x_i$$

Bias update:

$$b_k := b_k - \eta(\hat{y}_k - y_k)$$

$$b_k := b_k + \eta(y_k - \hat{y}_k)$$

Multiclass Update Intuition

If class k is the correct class:

$$y_k = 1$$

If prediction is low:

$$\hat{y}_k < 1$$

then:

$$y_k - \hat{y}_k > 0$$

So weights for the correct class increase. For wrong classes:

$$y_k = 0$$

If the model gives them high probability:

$$\hat{y}_k > 0$$

then:

$$y_k - \hat{y}_k < 0$$

So weights for wrong classes decrease.

Multiclass Update Example

Suppose actual class is:

$$y_2$$

So one-hot target:

$$\mathbf{y} = [0, 1, 0, 0, 0, 0]$$

Predicted probabilities:

$$\hat{\mathbf{y}} = [0.055, 0.090, 0.007, 0.100, 0.738, 0.010]$$

For class 2:

$$y_2 - \hat{y}_2 = 1 - 0.090 = 0.910$$

If:

$$\eta = 0.1$$

Update for Wrong Class Example

For class 5, the model predicted high probability:

$$\hat{y}_5 = 0.738$$

But actual class is not class 5:

$$y_5 = 0$$

So:

$$y_5 - \hat{y}_5 = 0 - 0.738 = -0.738$$

Update:

$$w_{5i} := w_{5i} + 0.1(-0.738)x_i$$

$$w_{5i} := w_{5i} - 0.0738x_i$$

Binary vs Multiclass Logistic Regression

Topic	Binary Logistic	Multiclass Logistic
Classes	2	K
Scores	One score z	K scores
Activation	Sigmoid	Softmax
Output	One probability	Probability distribution
Loss	Binary cross-entropy	Multiclass cross-entropy
Prediction	Threshold 0.5	Maximum probability

Key Idea

Softmax is the multiclass version of sigmoid-style probability conversion.

Stochastic, Batch, and Mini-Batch Processing

Method	Update Uses	Comment
Stochastic GD	One example	Fast but noisy
Batch GD	All examples	Stable but slow
Mini-batch GD	Small group	Common in practice

For mini-batch binary logistic regression:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

Meaning

Mini-batch gradient is the average of gradients from examples in the batch.

Matrix Form: Binary Logistic Regression

For m examples and f features:

$$X \in \mathbb{R}^{m \times f}$$

$$\mathbf{w} \in \mathbb{R}^{f \times 1}$$

$$\mathbf{z} = X\mathbf{w} + b$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{z})$$

Gradient:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y})$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

Matrix Form: Multiclass Logistic Regression

For m examples, f features, and K classes:

$$X \in \mathbb{R}^{m \times f}$$

$$W \in \mathbb{R}^{f \times K}$$

$$\mathbf{b} \in \mathbb{R}^{1 \times K}$$

Scores:

$$Z = XW + \mathbf{b}$$

$$Z \in \mathbb{R}^{m \times K}$$

Probabilities:

$$\hat{Y} = \textit{softmax}(Z)$$

Multiclass Matrix Gradient

Let:

$Y =$ one-hot target matrix

$$\hat{Y} = \text{softmax}(XW + \mathbf{b})$$

Then:

$$\frac{\partial J}{\partial W} = \frac{1}{m} X^T (\hat{Y} - Y)$$

Bias gradient:

$$\frac{\partial J}{\partial \mathbf{b}} = \frac{1}{m} \sum_{i=1}^m (\hat{Y}^{(i)} - Y^{(i)})$$

Update:

$$W := W - \eta \frac{\partial J}{\partial W}$$

$$\mathbf{b} := \mathbf{b} - \eta \frac{\partial J}{\partial \mathbf{b}}$$

Common Student Mistakes

- 1 Confusing z and $\hat{y}$.
- 2 Forgetting sigmoid is for binary classification.
- 3 Forgetting softmax probabilities sum to 1.
- 4 Using sigmoid independently for multiclass single-label classification.
- 5 Forgetting one-hot encoding in multiclass loss.
- 6 Updating only the correct class weights.
- 7 Forgetting that wrong class weights also decrease.
- 8 Mixing $(\hat{y} - y)$ and $(y - \hat{y})$ signs.

Final Summary

- Gradient descent minimizes a loss function.
- Regression example used SSR loss.
- Binary logistic regression uses sigmoid.
- Binary logistic loss is binary cross-entropy.
- Multiclass logistic regression uses softmax.
- Multiclass loss is:

$$L = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- Softmax gradient is:

$$\frac{\partial L}{\partial w_{ki}} = (\hat{y}_k - y_k)x_i$$

Gradient Descent + Sigmoid/Softmax + Cross Entropy = Logistic Regression Training